BATTLESHIP MULTIPLAYER

BÁO CÁO GIỮA KỲ

Môn Lập trình mạng căn bản (NT106) · UIT

Hoàng · Đức · Dũng

Mục Lục Nội Dung

Giới Thiệu

Cơ Sở Lý Thuyết

Phân Tích Và Thiết Kế

Demo

Tài Liệu Tham Khảo

Danh Mục Từ Viết Tắt

Viết tắt	Ý nghĩa
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
NAT	Network Address Translation
JSON	JavaScript Object Notation
DTO	Data Transfer Object
AES	Advanced Encryption Standard
CBC	Cipher Block Chaining
IV	Initialization Vector

Viết tắt	Ý nghĩa
ORM	Object-Relational Mapping
EF Core	Entity Framework Core
ERD	Entity Relationship Diagram
REST	Representational State Transfer
RLS	Row Level Security
UI	User Interface
GDI+	Graphics Device Interface Plus
NT106	Lập trình mạng căn bản

Giới Thiệu Đề Tài

Battleship (Bắn tàu) nhiều người chơi qua mạng

- Hai người chơi, mỗi người một lưới 10×10 và một đội tàu.
- Hai bên **lần lượt** gọi tọa độ để bắn; mỗi phát bắn là **trúng** hoặc **hụt**.
- Ai đánh chìm hết tàu đối phương trước thì thắng.
- Vị trí tàu của đối phương luôn bị **che** (sương mù).



Vì sao chọn đề tài này cho NT106

Trò chơi theo lượt, dữ liệu nhỏ, nhưng đủ các bài toán mạng thật: kết nối, đồng bộ trạng thái, chống gian lận, ngắt kết nối giữa chừng.

Giải Thuật Liên Quan (1/2): Mạng Và Đồng Thời

Giải thuật	Dùng ở	Ý tưởng chính
Đọc khung theo độ dài (length-prefix)	`PacketFraming`	Đọc đủ 4 byte độ dài N, rồi **lặp đọc** tới khi đủ N byte mới giải mã. Giải quyết dính gói, vỡ gói của TCP.
Hàng đợi FIFO ghép cặp	`MatchmakingQueue`	`ConcurrentQueue` + một worker nền: cứ có ≥ 2 người thì lấy **hai người đầu hàng** ghép thành phòng, ai vào trước được ghép trước. Kiểm tra còn kết nối ngay trước khi ghép.
Hủy tìm trận kiểu "đánh dấu"	`MatchmakingQueue`	Hàng đợi không xóa được phần tử ở giữa, nên khi hủy chỉ **xóa tên khỏi bảng băm** những người còn chờ; lúc lấy ra mà không thấy tên thì bỏ qua. Hủy mất $O(1)$.
Định tuyến gói theo kiểu	`PacketRouter`	Tra bảng băm `Type` $\rightarrow$ `Handler`, mất $O(1)$. Thêm tính năng chỉ cần đăng ký thêm một dòng.
Mỗi Client một Task	`TcpServer`	`async/await`: chờ mạng không chặn luồng; `SemaphoreSlim` bảo đảm mỗi Client chỉ có một Task ghi vào stream.

Giải Thuật Liên Quan (2/2): Bảo Mật Và Luật Chơi

Giải thuật	Dùng ở	Ý tưởng chính
AES-128-CBC	`CryptoHelper`	Mã hóa khối đối xứng, **IV ngẫu nhiên mỗi gói**; khóa 16 byte lấy từ SHA-256 của mật khẩu chung.
BCrypt	`AuthHandler`	Băm một chiều có **salt** và hệ số chi phí 11 (chuỗi lưu dạng `$2a$11$...`), cố tình chậm để chống dò mật khẩu.
Giới hạn tần suất (cửa sổ cố định)	`LoginRateLimiter`	Đếm số lần sai **theo IP và theo tên**; đủ 5 lần thì khóa 60 giây. Đăng ký tối đa 5 lần mỗi 10 phút mỗi IP.
Xử lý lượt bắn (thiết kế)	Turn Engine	Kiểm tra đúng lượt và ô chưa bắn, tra bản đồ theo ô trong $O(1)$; đếm số ô tàu còn lại để biết chìm hay thắng. **Trúng thì giữ lượt, hụt thì đổi lượt**; quá 30 giây tự đổi lượt.
Radar 3x3 (thiết kế)	Turn Engine	Duyệt 9 ô quanh ô được chọn và đếm ô có tàu: $O(9) = O(1)$. Chỉ trả về con số, không lộ vị trí.

Hiện Trạng: Các Game Battleship Trực Tuyến Có Sẵn

Tiêu chí	Battleship (battleship-game.org) [9]	Hasbro's BATTLESHIP (Steam) [10]	Fleet Battle (di động) [11]	Đồ án của nhóm
Nền tảng	Web, chạy trên trình duyệt	PC (Steam)	Android, iOS	Windows (WinForms)
Chơi qua mạng	2 người; chọn bạn bè hoặc đối thủ ngẫu nhiên	Online PvP, chế độ Classic và Commanders	Ghép trận nhanh với người chơi khắp nơi, có giải đấu	Tạo/vào phòng bằng ID, tìm trận ngẫu nhiên
Chat	Có, chat với đối thủ	Không thấy nêu	Có, chat chung và theo ngôn ngữ	Chat sảnh (chat trong trận: đang làm)
Bảng xếp hạng	Không thấy nêu	Không thấy nêu	Có	Có (REST Supabase)
Chi phí	Chạy ngay trên web	Trả phí (khoảng 2,5 – 5 USD tùy nơi bán)	Có quảng cáo và mua trong ứng dụng	Miễn phí
Tầng mạng	Chưa thấy tài liệu công bố	Chưa thấy tài liệu công bố	Chưa thấy tài liệu công bố	**Tự xây trên TCP thô, có tài liệu giao thức**

Thông tin lấy từ trang giới thiệu của từng sản phẩm (truy cập 09/11/2026), nhóm chưa tự cài đặt để kiểm chứng; "không thấy nêu" nghĩa là trang giới thiệu không đề cập.

Cái Đã Có, Cái Chưa Có Và Lý Do Chọn Đề Tài

Các sản phẩm trên đã làm được

- Chơi **trực tuyến thật** với người lạ hoặc bạn bè.
- Có chat, bảng xếp hạng, giải đấu (tùy sản phẩm).
- Chạy ổn định ở quy mô lớn.

Chưa phục vụ mục tiêu học tập mạng

- **Không xem được tầng mạng**: giao thức, cách đóng gói, cách chống gian lận.
- Một số sản phẩm **trả phí** hoặc có **quảng cáo**.
- Không thể mở rộng hay thử nghiệm tính năng mới.

Vì sao nhóm tự xây

- Môn NT106 yêu cầu làm việc **trực tiếp với socket**, nên cần tự cài giao thức.
- Battleship theo lượt: luật đơn giản, nhưng đủ các vấn đề mạng thật (đồng bộ trạng thái, chống gian lận, rớt kết nối).
- Có chỗ để thêm điểm riêng: **Radar**, **gửi kết quả trận ra Discord/Telegram**, **CSDL trên cloud**.

Đối Tượng Áp Dụng Và Đầu Ra Mong Đợi

Đối tượng áp dụng

- **Người chơi** muốn chơi Battleship qua mạng.
- **Người học lập trình mạng**: dùng làm ví dụ về socket, giao thức, bảo mật.
- **Người vận hành** (nhóm): chạy Server, quản lý dữ liệu.

Vai trò trong nhóm

Đức	Server, phòng, luật trận
Dũng	Giao diện, cơ sở dữ liệu
Hoàng	Giao thức, mạng, bảo mật

Đầu ra

1. **Server** TCP bất đồng bộ (C# .NET 8).
2. **Client** WinForms chơi được từ đăng nhập đến hết trận.
3. **Cơ sở dữ liệu** trên cloud (PostgreSQL) kèm REST đọc bảng xếp hạng.
4. **Giao thức mạng** có tài liệu: khung gói tin, bắt tay, mã hóa.
5. Báo cáo, slide và bản demo.

Mục Tiêu Đề Tài

Mục tiêu chung

Xây dựng game Battleship hai người chơi qua mạng, **tự cài giao thức trên TCP thô**, an toàn và chạy được giữa hai mạng khác nhau.

Mục tiêu cụ thể (kỹ thuật)

1. Giao thức riêng: khung gói 4 byte, bắt tay, định tuyến.
2. Server xử lý nhiều người chơi đồng thời, không sập khi Client rớt mạng.
3. **Server nắm luật**, chống gian lận.
4. Bảo mật: AES-128, BCrypt, khóa dò mật khẩu.

Mục tiêu cụ thể (sản phẩm)

1. Đăng ký, đăng nhập, phòng, ghép trận, chat.
2. Chơi trọn một trận: xếp tàu, bắn theo lượt, thắng thua.
3. Lưu lịch sử và bảng xếp hạng trên cloud.
4. Tính năng riêng: Radar, thông báo kết quả ra Discord/Telegram.

Phạm Vi Đề Tài

Trong phạm vi

- Tài khoản: đăng ký, đăng nhập, trạng thái Online/Offline.
- Sảnh chờ: chat, danh sách phòng, tạo/vào phòng bằng ID.
- Ghép trận tự động (Matchmaking).
- Xếp tàu, bắn theo lượt, sương mù, xử thắng khi đối thủ mất kết nối.
- Lưu lịch sử trận, bảng xếp hạng.
- Bảo mật: AES-128, BCrypt, khóa đăng nhập.
- Tính năng đặc biệt: **Radar** và **gửi kết quả trận ra Discord/Telegram**.

Ngoài phạm vi

- Chế độ khán giả (Spectator).
- Đối thủ máy (AI), bản di động, đồ họa 3D.
- Thanh toán, hệ thống bạn bè nâng cao.

Nền tảng

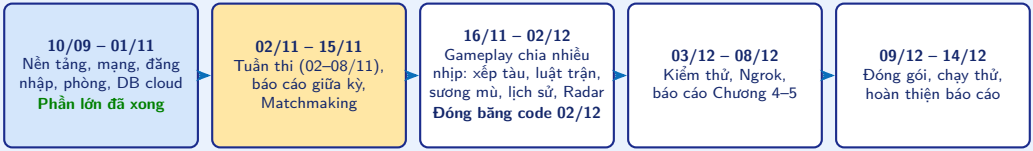
Ứng dụng desktop **Windows** (WinForms, C# .NET 8). Chạy trên máy cá nhân; demo giữa hai mạng khác nhau bằng Ngrok.

Tính Năng Và Tiến Độ Giữa Kỳ

Nhóm	Tính năng	Trạng thái
Tài khoản	Đăng ký, đăng nhập (BCrypt, khóa 5 lần sai), trạng thái Online/Offline	**Xong**
Phòng	Tạo/vào/rời phòng bằng ID	**Xong**
Phòng	Kick thành viên (chủ phòng)	**Dở dang**
Ghép trận	Tìm trận ngẫu nhiên (Matchmaking)	**Xong**
Giao tiếp	Chat sảnh	**Xong**
Giao tiếp	Chat trong trận	**Chưa**
Gameplay	Xếp tàu, bắn theo lượt, sương mù, tự xử thắng	**Chưa**
Thống kê	Bảng xếp hạng (REST Supabase)	**Xong**
Thống kê	Lưu lịch sử đấu	**Dở dang**
Bảo mật	Mã hóa AES-128 + BCrypt + giới hạn đăng ký	**Xong**
Dữ liệu	SQLite hoặc PostgreSQL (Supabase) qua EF Core	**Xong**
Đặc biệt	Radar, gửi kết quả ra Discord/Telegram	**Chưa**

Dở dang: đã có mã nhưng chưa nối đủ vào luồng chơi. **Chưa**: bắt đầu sau tuần thi, theo lịch ở slide kế tiếp.

Timeline Cập Nhật Đến Hết Học Kỳ



Sẵn sàng báo cáo từ **15/12/2026**; buổi học cuối 25/12/2026

Ghi chú

Việc then chốt là **logic trận đấu trên Server** (nhận Ready, xếp tàu, bắn, kết thúc). Các mục sương mù, lịch sử trận, Radar và webhook đều phụ thuộc vào mục này.

Mạng: Client–Server, TCP Và Khung Gói Tin

Client–Server, Server nắm luật

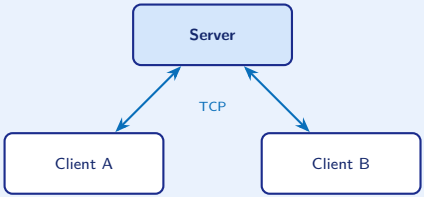
Client chỉ gửi **yêu cầu** và hiển thị **kết quả**; Server giữ trạng thái trận, nên Client không tự báo trúng/hụt hay nhìn bàn đối thủ.

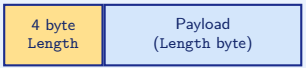
Vì sao TCP [1] mà không phải UDP

TCP bảo đảm **đến nơi, đúng thứ tự, không mất gói**: một lệnh bắn bị mất hay đảo thứ tự sẽ làm sai cả ván.

Vấn đề của TCP và cách giải

TCP là **dòng byte**, gói có thể dính hoặc vỡ. Mỗi gói đi trước bởi **4 byte độ dài**; bên nhận đọc đủ mới giải mã.





Một khung gói tin

Xử Lý Đồng Thời Và Giao Thức Ứng Dụng

Bất đồng bộ, mỗi Client một Task

- `async/await`: chờ mạng **không chặn luồng**; một Client chậm không làm kẹt Server.
- `ConcurrentDictionary`: danh sách phiên Online an toàn khi nhiều Task cùng sửa.
- `SemaphoreSlim(1,1)`: một Client chỉ có **một Task ghi** vào stream, tránh vỡ khung gói.

Bài học của nhóm

Ở Client, hai nơi cùng đọc một stream làm vỡ khung gói. Cách sửa: chỉ **một vòng lặp nền** được đọc stream.

Gói tin JSON [2] + Bắt tay + Router

- Gói = `Type` + `Payload` (DTO dạng **JSON**).
- **Bắt tay**: Client gửi phiên bản giao thức, Server chấp nhận hoặc ngắt.
- `PacketRouter`: mỗi nhóm chức năng là một `IPacketHandler` đăng ký theo kiểu gói; thêm tính năng không cần sửa lõi mạng.

Handler	Xử lý
`AuthHandler`	đăng ký, đăng nhập
`RoomHandler`	phòng đấu
`ChatHandler`	chat sảnh
`MatchmakingHandler`	tìm trận

Bảo Mật: AES-128 Và BCrypt

AES-128-CBC [3] mã hóa đường truyền

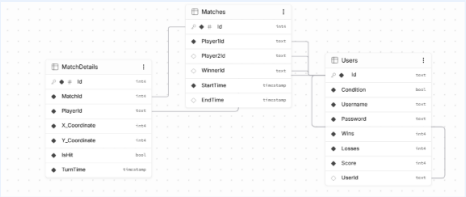
- Mã hóa **đối xứng**: cùng một khóa để mã hóa và giải mã.
- **IV ngẫu nhiên mỗi gói**, gửi kèm: cùng nội dung vẫn ra bản mã khác nhau.
- Khóa 16 byte lấy từ `SHA-256` của mật khẩu chung hai phía.

Hạn chế đã biết

Khóa chung nằm trong ứng dụng, chưa trao đổi khóa khi kết nối. Hướng nâng cấp: **TLS** hoặc trao đổi khóa Diffie–Hellman.

BCrypt [4] băm mật khẩu

- Băm **một chiều**, có **salt** ngẫu nhiên: hai người cùng mật khẩu vẫn ra hai chuỗi khác nhau.
- Cố tình **chậm** để kẻ tấn công dò mật khẩu tốn kém.
- Cơ sở dữ liệu không lưu mật khẩu rõ; chuỗi lưu dạng `$2a$11$...`



Dữ Liệu, Cloud Và Giao Diện: Tech Stack

EF Core [6]: SQLite hoặc PostgreSQL

- Viết lớp C#; EF tự sinh bảng và truy vấn (tham số hóa, chống SQL injection).
- Đổi hệ quản trị chỉ đổi **provider**: **SQLite** chạy offline, **PostgreSQL (Supabase)** dùng chung trên cloud. Chọn bằng biến môi trường `BATTLESHIP_DB`.

Supabase REST [8] và RLS [7]

PostgreSQL trên cloud, tự sinh REST API qua HTTPS. **RLS** khóa bảng: khóa công khai chỉ đọc view `leaderboard`, bảng `Users` (có mật khẩu băm) bị chặn.

Công nghệ	Dùng để
C# .NET 8	toàn bộ hệ thống
`System.Net.Sockets` [5]	TCP thô
`System.Text.Json`	đóng gói JSON
AES, BCrypt.Net	mã hóa, băm
EF Core, Npgsql	cơ sở dữ liệu
WinForms, GDI+	giao diện, vẽ bàn cờ
Supabase	cloud, REST

Lưu ý với WinForms

Dữ liệu từ Server về trên luồng nền, nhưng chỉ luồng giao diện được sửa điều khiển: phải dùng `Invoke` để tránh lỗi cross-thread.

Yêu Cầu Chức Năng Và Phi Chức Năng

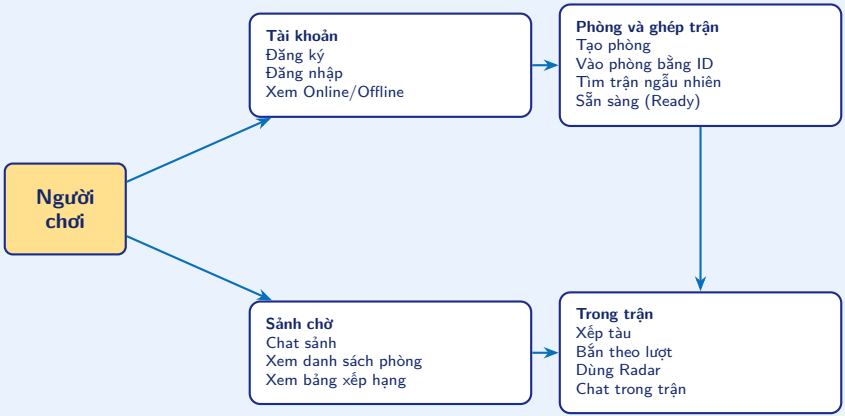
Yêu cầu chức năng

1. Đăng ký, đăng nhập, xem ai đang Online.
2. Chat ở sảnh chờ.
3. Tạo phòng, vào phòng bằng ID, tìm trận ngẫu nhiên.
4. Xếp tàu, sẵn sàng (Ready), bắn theo lượt.
5. Báo trúng/hụt, chìm tàu, kết thúc trận.
6. Lưu lịch sử trận, xem bảng xếp hạng.

Yêu cầu phi chức năng

- **Đúng luật**: Server quyết định, chống gian lận.
- **Bảo mật**: mã hóa gói tin, băm mật khẩu, chống dò mật khẩu.
- **Đồng thời**: nhiều người chơi, không nghẽn.
- **Bền**: Client rớt mạng không làm Server sập.
- **Dễ chạy**: có chế độ SQLite offline để demo.

Biểu Đồ Chức Năng Của Người Chơi



Mũi tên giữa các nhóm thể hiện thứ tự dùng. Server là bên xử lý mọi yêu cầu.

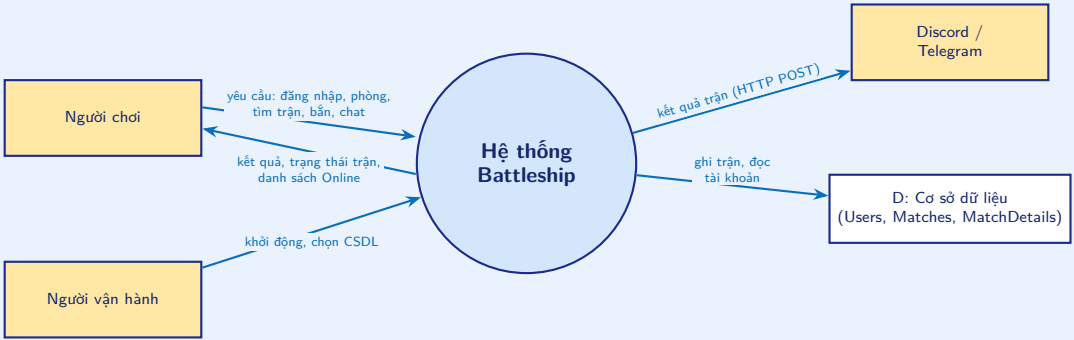
Các Vai Trò Và Phân Quyền

Chức năng	Người chơi	Chủ phòng	Người vận hành
Đăng ký, đăng nhập, xem Online	✓	✓	
Chat sảnh, xem bảng xếp hạng	✓	✓	
Tạo phòng, vào phòng, tìm trận	✓	✓	
Chơi trận (xếp tàu, bắn, Radar)	✓	✓	
Kick thành viên khỏi phòng		✓	
Chạy Server, chọn cơ sở dữ liệu, xem log			✓
Quản lý dữ liệu trên Supabase			✓

Ghi chú

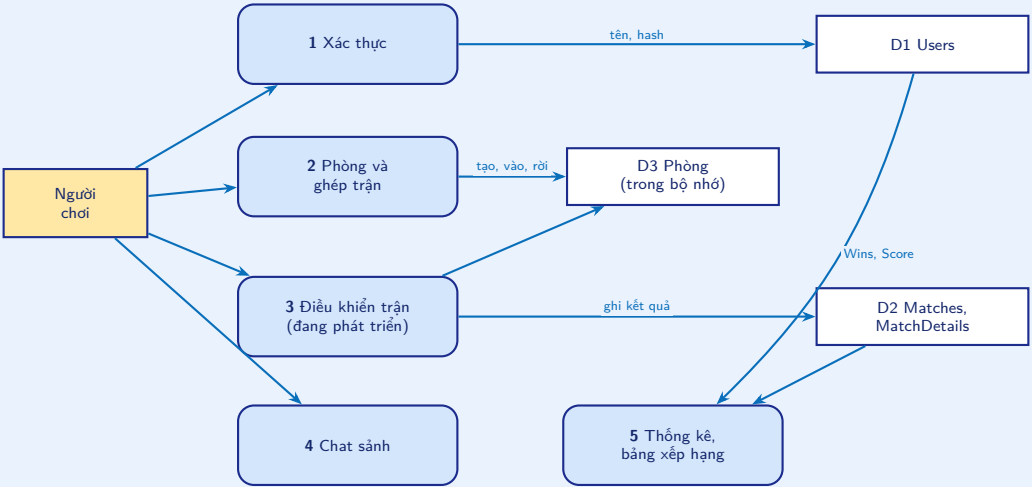
Chủ phòng là người tạo phòng (hoặc người đứng đầu hàng đợi khi ghép trận), có thêm quyền kick; Server kiểm tra quyền này, Client không tự quyết. **Người vận hành** là nhóm phát triển, thao tác trực tiếp trên Server và Supabase. Hệ thống **không có tài khoản admin** trong phạm vi đề tài.

Lược Đồ Dòng Dữ Liệu (DFD) Mức 0



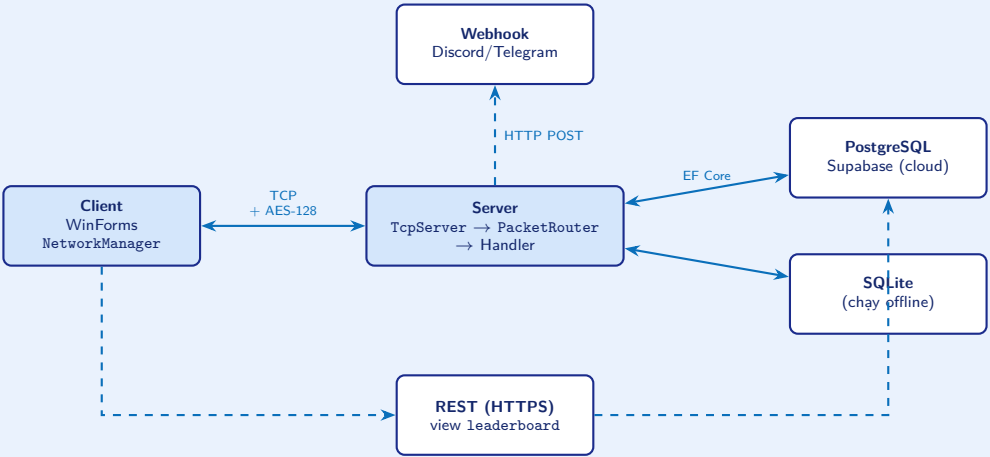
Người chơi còn đọc bảng xếp hạng trực tiếp qua REST (chỉ đọc). Webhook đang phát triển.

Lược Đồ Dòng Dữ Liệu (DFD) Mức 1



1, 2, 4 đã chạy. **3** (luật trận) và ghi **D2** đang phát triển. **5** đọc qua view `leaderboard`.

Kiến Trúc Tổng Thể



Nét liền: đường chính (TCP, cơ sở dữ liệu). Nét đứt: chỉ đọc bảng xếp hạng qua REST; webhook (đang phát triển).

Thiết Kế Giao Thức Gói Tin

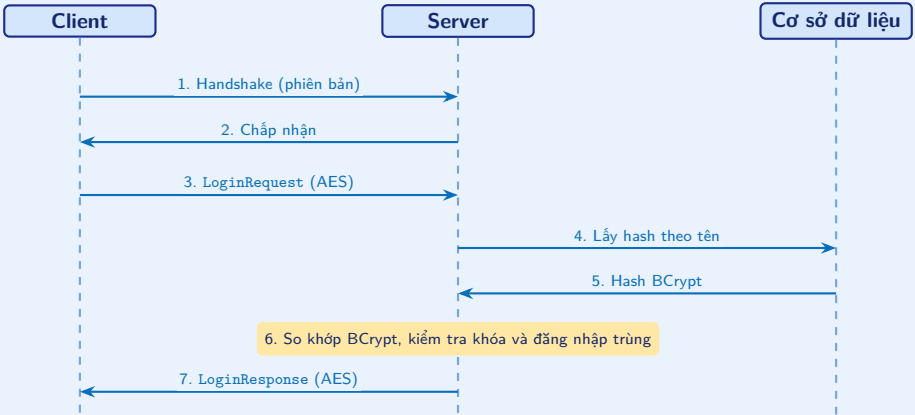
Cấu trúc một gói

`[4 byte độ dài][AES(JSON{Type, Payload})]`

- Type là kiểu liệt kê (PacketType).
- Payload là JSON của DTO tương ứng.
- Gói đầu tiên là Handshake (phiên bản giao thức).

Nhóm	Kiểu gói chính
Tài khoản	`LoginRequest/Response`, `RegisterRequest/Response`
Phòng	`CreateRoomRequest`, `JoinRoomRequest/Response`, `RoomUpdate`, `LeaveRoom`
Ghép trận	`MatchmakingJoin/CancelRequest`, `MatchFound`
Trận đấu	`PlaceShipsRequest`, `PlayerReady`, `GameStart`, `FireShotRequest`, `ShotResult`, `TurnChange`, `GameOver`
Khác	`ChatMessage`, `OnlineListUpdate`, `Ping/Pong`, `Error`

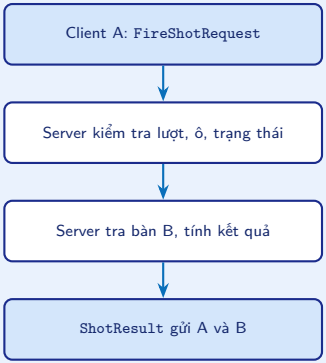
Luồng Dữ Liệu: Đăng Nhập



Luồng Dữ Liệu: Một Lượt Bắn (Thiết Kế)

Server-Authoritative

1. Client gửi `FireShotRequest` (tọa độ).
2. Server kiểm tra: **đúng lượt?** ô **chưa bắn?** trận **đang chơi?** Sai thì bỏ qua.
3. Server tra bàn của đối thủ, quyết định **Hit/Miss/Chìm**.
4. Server gửi `ShotResult` cho **cả hai**; **trúng thì giữ lượt, hụt thì** `TurnChange` (luật ở slide sau).
5. Chìm tàu cuối cùng: gửi `GameOver`, ghi kết quả vào cơ sở dữ liệu.



Đang phát triển

Logic trận trên Server (bước 2–5) chưa xong; xem lịch ở mục Giới thiệu.

Luật Chuyển Lượt (Đã Chốt)

Hành động	Kết quả từ Server	Lượt tiếp theo
Bắn tọa độ (X, Y)	**Trúng** (Hit) hoặc **chìm** (Sunk)	**Được bắn tiếp**: Server giữ nguyên `CurrentTurn`
Bắn tọa độ (X, Y)	**Hụt** (Miss)	**Đổi lượt** sang đối thủ
Bắn làm chìm tàu cuối cùng của đối phương	Hết tàu	Chuyển thẳng sang **GameOver**
Dùng **Radar**	Trả kết quả vùng quét	**Mất 1 lượt**: đổi lượt sang đối thủ ngay sau khi trả kết quả
Quá **30 giây** mỗi lượt	Hết giờ lượt hiện tại	Tự động đổi lượt sang đối thủ
Mất kết nối (`SocketException`)	Xử thua bên vi phạm	Chuyển sang **GameOver**

Nguyên tắc

Server nắm toàn bộ bàn 10×10 của hai bên; Client không biết vị trí tàu đối phương. Lệnh bắn ngoài lượt bị Server bỏ qua. Luật do Đức chốt, đã nằm trong thiết kế State Machine.

Lõi TCP Server Và Quản Lý Phiên

Mô hình bất đồng bộ (`TcpServer`)

- `TcpListener` lắng nghe; **mỗi kết nối chạy trên một Task riêng**.
- Danh sách Online quản lý bằng `Concurrent Dictionary`.
- Điều hướng gói tin qua `PacketRouter` theo giao diện `IPacketHandler`.

An toàn luồng và chống sập (`ClientSession`)

- Đọc trọn gói theo framing 4 byte (`ReadPacketAsync`).
- Khóa gửi `SemaphoreSlim(1,1)`: chặn nhiều Task cùng gửi làm đan xen byte.
- Client ngắt đột ngột: bắt `IOException/SocketException`, dọn phiên, không làm sập Server.

[CHƯA CÓ ẢNH]

`images/10-server-log.png`

Log khởi động TCP Server và kết nối Supabase

Quản Lý Phòng Đấu Đa Luồng (RoomManager)

Chống race condition

- **Danh sách phòng**: `Concurrent Dictionary<string, GameRoom>` an toàn khi tạo/xóa phòng đồng thời.
- **Bên trong phòng**: `lock(room.RoomLock)` chặn hai người cùng vào một phòng trống ở cùng một mili-giây.

Luồng gói tin phòng (RoomHandler)

- Đã hiện thực: `GetRoomListRequest`, `CreateRoomRequest`, `JoinRoomRequest`, `LeaveRoom`.
- Chủ phòng (Host) thoát thì phòng tự giải tán.
- Kick đã có hàm, **chưa nối vào Router**.

[CHƯA CÓ ẢNH]

`images/11-room-list.png`

Màn hình Join Room có ít nhất một phòng (tạo phòng bằng một Client rồi chụp Client kia)

Máy Trạng Thái Của Phòng Và Trận (`RoomState`)

5 trạng thái của `GameRoom`

- **WaitingForPlayers**: chờ đủ 2 người (đã code).
- **PlacementPhase**: tự chuyển khi người thứ hai vào phòng (đã code khung).
- **ReadyCheck**: chờ cả hai gửi đội hình tàu (đang phát triển).
- **TurnBasedPlay**: đánh theo lượt, có kiểm soát timeout (đang phát triển).
- **GameOver**: kết thúc, lưu thống kê và đóng phòng.

Quy tắc

Chỉ nhận lệnh bắn khi ở **TurnBasedPlay** và đúng lượt. Vào trận chỉ khi **cả hai** sẵn sàng.



Xanh: đã code. Còn lại: đang phát triển.

Thiết Kế Cơ Sở Dữ Liệu (ERD)



Một người chơi có nhiều trận; một trận có nhiều lượt bắn (**MatchDetails**). View `leaderboard` chỉ lấy tên, thắng, thua, điểm từ `Users`.

Thiết Kế Bảo Mật Nhiều Lớp

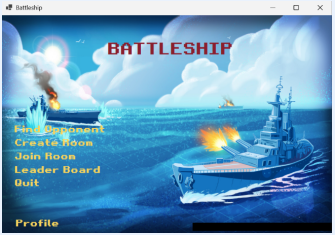
Lớp	Biện pháp	Chống lại
Đường truyền	AES-128-CBC, IV ngẫu nhiên mỗi gói	Nghe lén, đọc trộm gói tin
Mật khẩu	BCrypt (có salt), kiểm tra dù tên không tồn tại	Lộ mật khẩu nếu rò rỉ DB, dò tên tài khoản
Đăng nhập	Khóa 60 giây sau 5 lần sai (theo IP và theo tên)	Dò mật khẩu hàng loạt
Đăng ký	Tối đa 5 lần mỗi 10 phút mỗi IP	Tạo tài khoản rác
Phiên	Router chặn mọi gói (trừ đăng nhập/đăng ký) khi chưa đăng nhập; một tài khoản một phiên	Gửi lệnh khi chưa xác thực, đăng nhập trùng
Dữ liệu	RLS; khóa công khai chỉ đọc view bảng xếp hạng	Đọc trực tiếp bảng `Users` từ ngoài
Luật chơi	Server quyết định Hit/Miss	Client gian lận

Thiết Kế Giao Diện

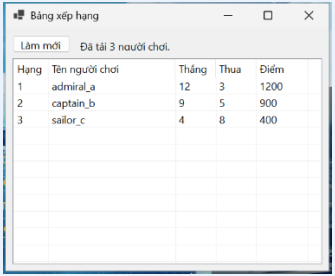
Đăng nhập



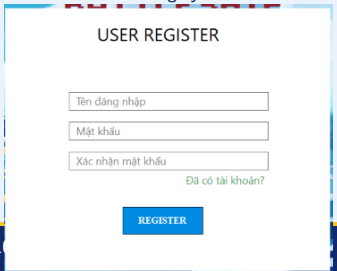
Sảnh chờ (Lobby)



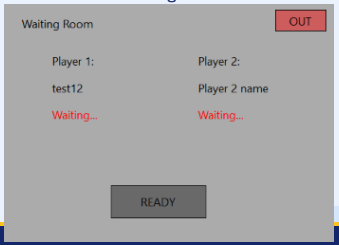
Bảng xếp hạng



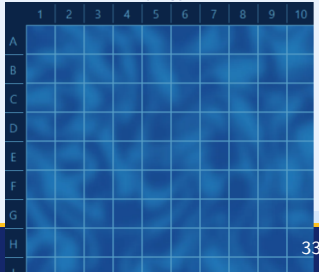
Đăng ký



Phòng chờ



Bàn cờ



Thiết Kế Tính Năng Đặc Biệt

Radar (quét vùng 3x3)

- Mỗi người dùng **một lần mỗi trận**, chọn một ô.
- Server đếm số ô tàu trong vùng 3×3 quanh ô đó và chỉ trả về **con số**, không nói ô nào.
- Dùng Radar thì **mất một lượt**: sau khi trả kết quả quét, Server đổi lượt sang đối thủ ngay.
- Server giữ cờ "đã dùng" và áp luật đúng lượt: Client không dùng được hai lần.

Gửi kết quả trận ra Discord/Telegram

- Khi trận kết thúc, Server gọi **HTTP POST** tới webhook.
- Tin nhắn xuất hiện ngay trên điện thoại: "A thắng B sau N lượt".
- Nối mạng game (TCP) với Internet (HTTP) và dịch vụ bên ngoài.
- Chạy ở Task riêng với thời hạn 5 giây; lỗi chỉ ghi log, không làm treo Server. URL webhook để trong biến môi trường.

Trạng thái

Đang ở bước thiết kế; hiện thực sau khi Server có logic trận (dự kiến 28/11 – 02/12).

Kịch Bản Demo

#	Việc làm và kết quả mong đợi	Hiện tại
1	Chạy Server: in “Đang dùng: Supabase” và “Đang lắng nghe tại cổng 8000”	Chạy
2	Đăng ký hai tài khoản; trên Supabase mật khẩu là chuỗi BCrypt	Chạy
3	Đăng nhập; sai 5 lần thì khóa 60 giây có đếm ngược	Chạy
4	Hai Client vào Lobby, chat sảnh, thấy nhau Online	Chạy
5	Cả hai bấm “Find Opponent”: Server ghép cặp, hai bên vào phòng chờ	Chạy
6	Mở bảng xếp hạng (REST Supabase)	Chạy
7	Chơi một trận: xếp tàu, bắn, Radar; kết quả hiện trên Discord	Chưa

Chạy trên cùng một máy (hai cửa sổ Client). Có phương án dự phòng chạy bằng SQLite khi mất mạng.

[CHƯA CÓ ẢNH]

`images/10-server-log.png`

Cửa sổ Server: dòng “Đang dùng: Supabase” và “Đang lắng nghe tại cổng 8000”

Tài Liệu Tham Khảo (IEEE) I

[1] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc793

[2] T. Bray, Ed., "The JavaScript Object Notation (JSON) Data Interchange Format," IETF, RFC 8259, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8259

[3] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001. [Online]. Available: https://csrc.nist.gov/pubs/fips/197/final

[4] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf. (FREENIX Track)*, Monterey, CA, USA, Jun. 1999, pp. 81–91.

[5] Microsoft, "System.Net.Sockets Namespace," .NET API Documentation, Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets. Accessed: Nov. 9, 2026.

[6] Microsoft, "Entity Framework Core," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/. Accessed: Nov. 9, 2026.

[7] Supabase, "Row Level Security," Supabase Docs. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security. Accessed: Nov. 9, 2026.

[8] Supabase, "REST API," Supabase Docs. [Online]. Available: https://supabase.com/docs/guides/api. Accessed: Nov. 9, 2026.

[9] "Battleship: online game for 2 players," Chrome Web Store. [Online]. Available: https://chrome.google.com/webstore/detail/battleship/hjgmfhnanfbghmpcbdfgpigcgdbaggfm. Accessed: Nov. 9, 2026.

[10] Marmalade Game Studio, "Hasbro's BATTLESHIP," Steam. [Online]. Available: https://store.steampowered.com/app/893050/Hasbros_BATTLESHIP/. Accessed: Nov. 9, 2026.

[11] "Fleet Battle: Sea Battle game," Apple App Store. [Online]. Available: https://apps.apple.com/us/app/id904275738. Accessed: Nov. 9, 2026.

CẢM ƠN CÔ VÀ CÁC BẠN ĐÃ LẮNG NGHE

Hỏi đáp (Q&A)

Hoàng · Đức · Dũng